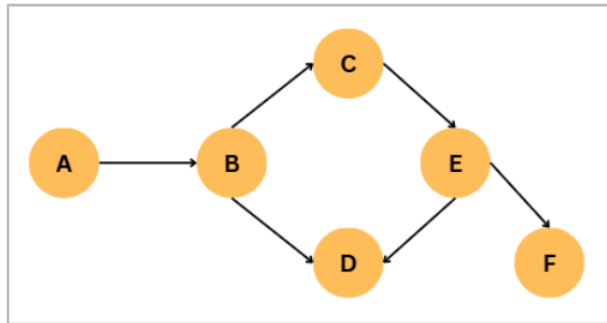


Theories

1. Find the vertex and edges of this graph



$V = \{A, B, C, D, E, F\}$

$E = \{ \{A,B\}, \{B,C\}, \{C,E\}, \{D,B\}, \{E,D\}, \{E,F\} \}$

2.

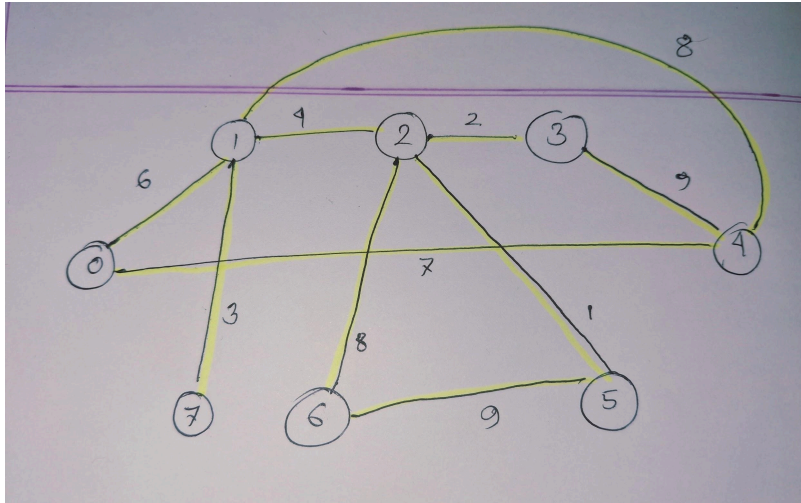
	0	1	2	3	4	5	6	7
0	0	6	0	0	7	0	0	0
1	6	0	4	0	8	0	0	3
2	0	4	0	2	0	1	8	0
3	0	0	2	0	9	0	0	0
4	7	8	0	9	0	0	0	0
5	0	0	1	0	0	0	9	0
6	0	0	8	0	0	9	0	0
7	0	3	0	0	0	0	0	0

The adjacency matrix shown above represents a weighted graph

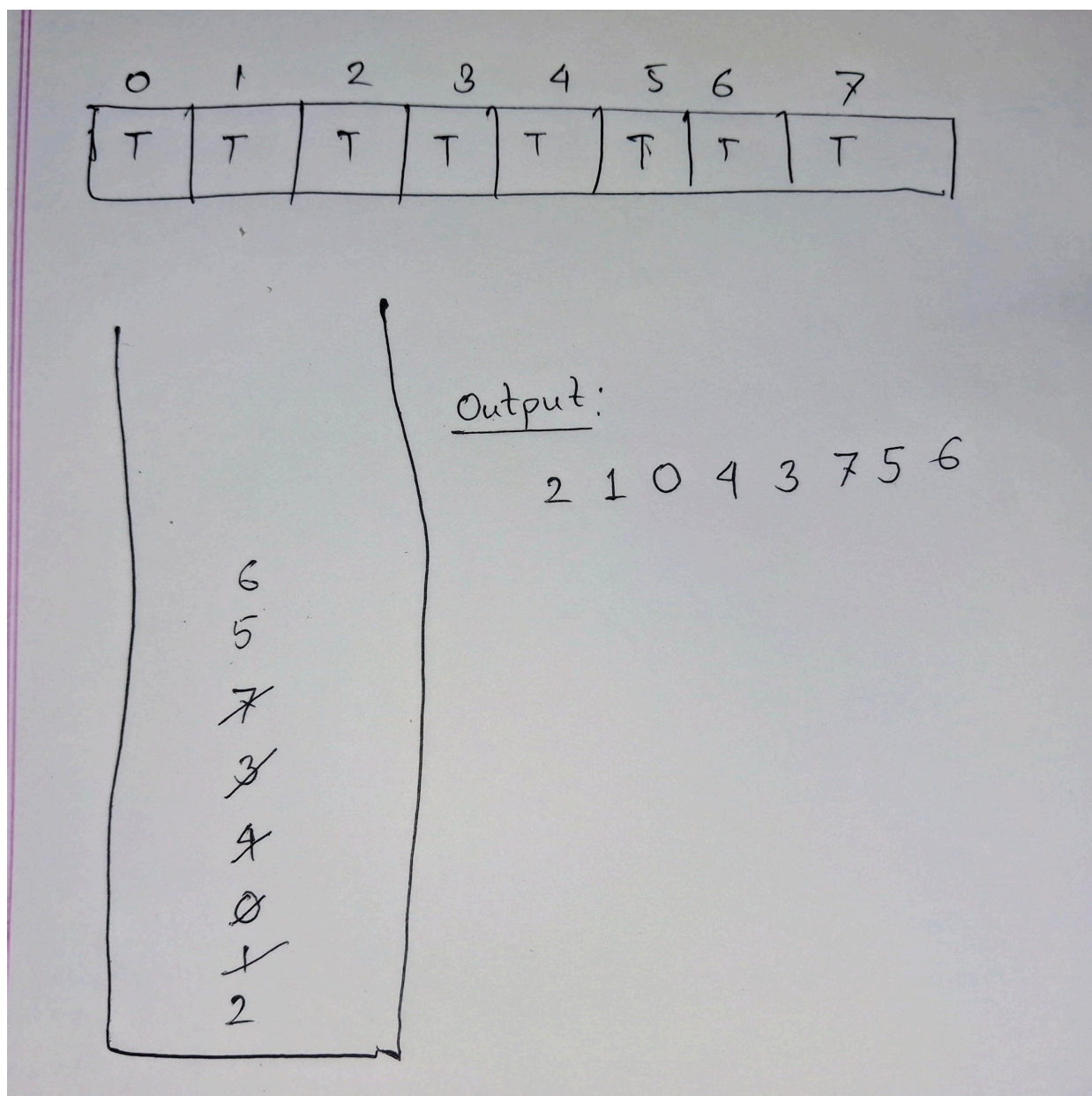
- Draw the Graph. [2 marks]
- Show the traversal sequence by performing a special Depth First Search (DFS) traversal of the graph starting from vertex 2. The DFS is special because when a vertex has multiple unvisited neighbors, the traversal must visit the neighbors in ascending order of their edge weights, meaning the one with the smallest weight will be chosen to visit first. [2 mark]

Note: You do not need to write any code for this question, just show the simulation.

a.



b. DFS Sequence



3. You are given the adjacency list of an undirected, unweighted graph. If the total length of all linked lists is 20, how many edges are there? What if the graph is directed?

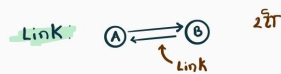
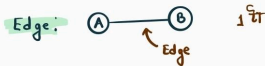
Ans:

Undirected graph: 10 edges

Directed graph: 20 edges

Explanation

① Undirected



② Directed



4. Advantage of Adjacency List over Adjacency Matrix

Adjacency List is more space efficient. Adjacency List is best when the graph has many vertices but few edges.

- It stores only the existing edges of the graph.
- Space required is $O(V + E)$, where V = vertices and E = edges.
- In sparse graphs (few edges), this saves a lot of memory compared to a matrix, which stores many useless 0s.

5. Advantage of Adjacency Matrix over Adjacency List

Adjacency Matrix allows faster edge checking. Adjacency Matrix is useful when the graph is dense or when frequent edge lookups are needed.

- Checking whether an edge exists between two vertices takes $O(1)$ time.
- You directly look at $\text{matrix}[u][v]$ instead of searching through a list.

6. In any directed, unweighted graph, the sum of all vertex degrees (in + out) is always even.

True.

(Each edge contributes 1 to in-degree and 1 to out-degree, total 2.)

7. In a graph stored as adjacency lists, determining whether edge (u, v) exists takes $O(1)$ time in the worst case.

False.

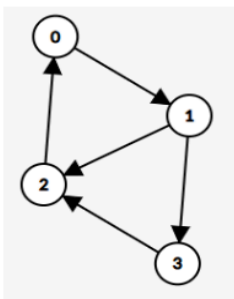
Correct answer: In the worst case, it takes $O(\deg(u))$ time, because we may need to scan the entire adjacency list of vertex u .

8.

Answer questions Q1, Q2, and Q3 using the following adjacency matrix representing a graph:

0	1	0	0
0	0	1	1
1	0	0	0
0	0	1	0

Graph:



Q1. Is there any cycle in the mentioned graph?

a) **Yes** b) No c) Because e.g., $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$

Q2. What's the in-degree of Vertex 2 in the mentioned graph?

a) 1 b) **2** c) 3 d) 4 e) Because incoming to 2: $1 \rightarrow 2$ and $3 \rightarrow 2$

Q3. What is the shortest path length from vertex 0 to vertex 2?

a) 1 b) **2** c) 3 d) 4 e) Because Shortest path: $0 \rightarrow 1 \rightarrow 2 \Rightarrow$ length 2

9. What is the time complexity of BFS using an adjacency list?

Ans: $O(V+E)$

10. What is the time complexity of DFS using an adjacency matrix?

Ans: $O(V^2)$

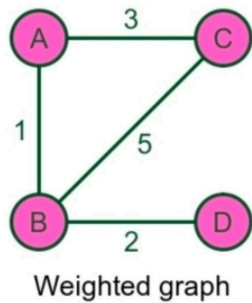
11. In which graph representation is edge existence checking fastest?

Ans: Adjacency Matrix

(Time complexity: $O(1)$)

Adjacency Matrix

Add to Adjacency Matrix



	A	B	C	D
A	0	1	3	0
B	1	0	5	2
C	3	5	0	0
D	0	2	0	0

Row = outgoing edges

Column = incoming
edges

```
private static void addEdge(int[][] matrix, int from, int to, int weight) {  
    matrix[from][to] = weight;  
    matrix[to][from] = weight; //If undirected  
}
```

Count number of edges (Adjacency Matrix)

Works for:

- directed / undirected [Undirected hole divide by 2]
- weighted / unweighted

```
public static int countEdges(int[][] graph, boolean directed) {  
    int edges = 0;  
    int n = graph.length;  
  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            if (graph[i][j] != 0) {  
                edges++;  
            }  
        }  
    }  
}
```

```

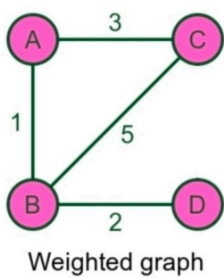
}

// In undirected graph, each edge is counted twice
if (!directed) {
    edges = edges / 2;
}

return edges;
}

```

Vertex with maximum degree (Undirected, Unweighted)



	A	B	C	D
A	0	1	3	0
B	1	0	5	2
C	3	5	0	0
D	0	2	0	0

Output:

B has the maximum degree. B has 3 degrees.
So the output is 3.

```

public static int maxDegree(int[][] graph) {
    int n = graph.length;
    int maxDegree = 0;

    for (int i = 0; i < n; i++) {
        int degree = 0;

        for (int j = 0; j < n; j++) {
            if (graph[i][j] != 0) {
                degree++;
            }
        }
    }
}

```

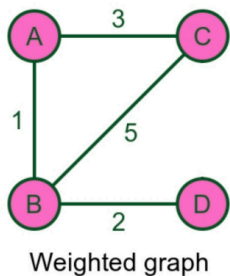
```

    if (degree > maxDegree) {
        maxDegree = degree;
    }
}

return maxDegree;
}

```

Vertex with maximum sum of edge weights



	A	B	C	D
A	0	1	3	0
B	1	0	5	2
C	3	5	0	0
D	0	2	1	0

Output:

Weight of vertex A= $0+1+3+0=4$

Weight of vertex B= $1+0+5+2=8$

Weight of vertex C= $3+5+0+=8$

Weight of vertex D= $0+2+1+0=3$

So the output is 8

```

public static int maxOutgoingWeight(int[][] graph) {
    int n = graph.length;
    int maxSum = 0;

    for (int i = 0; i < n; i++) {
        int sum = 0;

        for (int j = 0; j < n; j++) {
            if (graph[i][j] != 0) {
                sum = sum + graph[i][j];
            }
        }

        if (sum > maxSum) {

```

```

        maxSum = sum;
    }
}

return maxSum;
}

```

Minimum incoming edge weight (directed)

```

static int minIncomingWeightVertex(int[][] g) {
    int min = 0;
    int vertex = -1;

    for (int i = 0; i < g.length; i++) {
        int w = valueFromMatrix(g, i, "incoming", true);
        if (w < min) {
            min = w;
            vertex = i;
        }
    }
    return vertex;
}

```

//helper

```

static int valueFromMatrix(int[][] g, int v, String direction, boolean weighted) {
    int value = 0;

    if (direction.equals("outgoing")) {
        for (int j = 0; j < g.length; j++) {
            if (g[v][j] != 0) { // row v, fixed
                if (weighted) value += g[v][j];
                else value++;
            }
        }
    }
}

```

```

}

if (direction.equals("incoming")) {
    for (int i = 0; i < g.length; i++) {
        if (g[i][v] != 0) { // column v, fixed
            if (weighted) value += g[i][v];
            else value++;
        }
    }
}

return value;
}

```

Maximum |in-degree – out-degree|

```

static int maxDegreeDifference(int[][] g) {
    int max = Integer.MIN_VALUE;
    int vertex = -1;

    for (int i = 0; i < g.length; i++) {
        int in = valueFromMatrix(g, i, "incoming", false);
        int out = valueFromMatrix(g, i, "outgoing", false);
        int diff = Math.abs(in - out);

        if (diff > max) {
            max = diff;
            vertex = i;
        }
    }
    return vertex;
}

```

```
//helper
static int valueFromMatrix(int[][] g, int v, String direction, boolean weighted) {
    int value = 0;

    if (direction.equals("outgoing")) {
        for (int j = 0; j < g.length; j++) {
            if (g[v][j] != 0) { // row v, fixed
                if (weighted) value += g[v][j];
                else value++;
            }
        }
    }

    if (direction.equals("incoming")) {
        for (int i = 0; i < g.length; i++) {
            if (g[i][v] != 0) { // column v, fixed
                if (weighted) value += g[i][v];
                else value++;
            }
        }
    }

    return value;
}
```

Same-degree pairs (undirected)

```
static int countSameDegreePairs(int[][] g) {
    int count = 0;

    for (int i = 0; i < g.length; i++) {
        for (int j = i + 1; j < g.length; j++) {
            if (valueFromMatrix(g, i, "outgoing", false) == valueFromMatrix(g, j, "outgoing",
false)) {
                count++;
            }
        }
    }
}
```

```

    }
}
return count;
}

```

```

//helper
static int valueFromMatrix(int[][] g, int v, String direction, boolean weighted) {
    int value = 0;

    if (direction.equals("outgoing")) {
        for (int j = 0; j < g.length; j++) {
            if (g[v][j] != 0) { // row v, fixed
                if (weighted) value += g[v][j];
                else value++;
            }
        }
    }

    if (direction.equals("incoming")) {
        for (int i = 0; i < g.length; i++) {
            if (g[i][v] != 0) { // column v, fixed
                if (weighted) value += g[i][v];
                else value++;
            }
        }
    }

    return value;
}

```

Directed to Undirected(Adjacency Matrix)

```

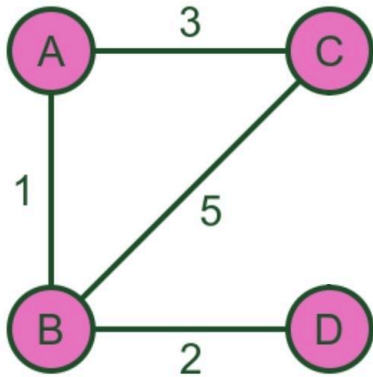
public void convertToUndirected(){
    for(int i=0; i<adjMatrix.length;i++){
        for(int j=0; j<adjMatrix.length;j++){
            if(adjMatrix[i][j]!=0){

```

```
        adjMatrix[j][i]=adjMatrix[i][j];  
    }  
}  
}
```


Adjacency List

Add Edge (Adjacency List)

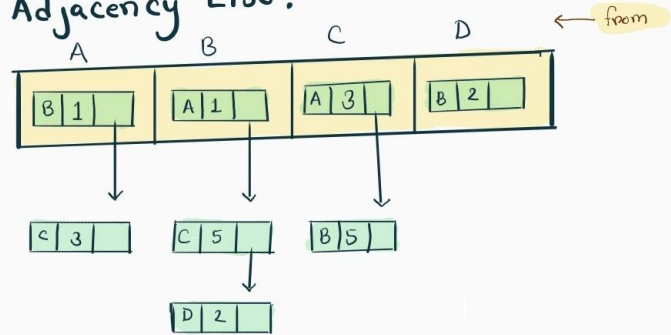


Weighted graph

Edge Node:

to V/ Destination	weight	Next
-------------------------------------	--------	------

Adjacency List:



//Edge class

```
class EdgeNode {  
    int toV;  
    int weight;  
    EdgeNode next;
```

```
    EdgeNode(Object toV, int weight) {  
        this.toV = toV;  
        this.weight = weight;  
        this.next = null;  
    }  
}
```

//Add method

```
static void addEdge(EdgeNode[] adjList, int from, int to, int weight) {
```

// forward edge

```
    EdgeNode newNode = new EdgeNode(to, weight);
```

```
    if (adjList[from] == null) {  
        adjList[from] = newNode;  
    } else {
```

```

    EdgeNode temp = adjList[from];
    while (temp.next != null) {
        temp = temp.next;
    }
    temp.next = newNode;
}

```

// reverse edge (If undirected)

```

EdgeNode revNode = new EdgeNode(from, weight);

```

```

if (adjList[to] == null) {
    adjList[to] = revNode;
} else {
    EdgeNode temp = adjList[to];
    while (temp.next != null) {
        temp = temp.next;
    }
    temp.next = revNode;
}
}

```

Vertex with Maximum Degree

```

public static int maxDegree(edgeNode[] adjList) {

    int maxDegree = 0;

    for (int i = 0; i < adjList.length; i++) {
        int degree = 0;

        edgeNode cur = adjList[i];
        while (cur != null) {
            degree++;
            cur = cur.next;
        }
    }
}

```

```

    }

    if (degree > maxDegree) {
        maxDegree = degree;
    }
}

return maxDegree;
}

```

Vertex with Minimum Incoming Edge Weight

```

static int minIncomingWeightVertex(edgeNode[] adjList) {

    int min = 0;
    int vertex = -1;

    for (int v = 0; v < adjList.length; v++) {
        int sum = 0;

        for (int i = 0; i < adjList.length; i++) {
            edgeNode cur = adjList[i];
            while (cur != null) {
                if (cur.toV == v)
                    sum += cur.weight;
                cur = cur.next;
            }
        }

        if (sum < min) {
            min = sum;
            vertex = v;
        }
    }
    return vertex;
}

```

Directed to Undirected(Adjacency List)

```
public void convertToUndirected() {
    for (int i = 0; i < adjList.length; i++) {
        EdgeNode curr = adjList[i];
        while (curr != null) {
            // check if reverse exists
            EdgeNode curr2 = adjList[curr.toV];
            boolean found = false;
            while (curr2 != null) {
                if (curr2.toV == i) {
                    found = true;
                    break;
                }
                curr2 = curr2.next;
            }
            if (!found) {
                // add reverse edge
                EdgeNode newRev = new EdgeNode(i, curr.weight);
                newRev.next = adjList[curr.toV];
                adjList[curr.toV] = newRev;
            }
            curr = curr.next;
        }
    }
}
```

Lab Task[Not important]

Adjacency Matrix

```
public class Task0a {
    int[][] adjMatrix;
    int n=0;

    Task0a(int n){
        this.n=n;
        adjMatrix= new int[n][n];
    }

    public void addEdge(int sender, int receiver, int weight,
boolean undirected){
        adjMatrix[sender][receiver]= weight;
        if(undirected==true){
            adjMatrix[receiver][sender]=weight;
        }
    }

    public void printWeight(){
        System.out.println("The weights are: ");
        for(int i=0; i<n; i++){
            for(int j=0; j<n; j++){
                System.out.print(adjMatrix[i][j]+ " ");
            }
            System.out.println();
        }
    }
}
```

```
public class Tester {
    public static void main(String[] args){
        Task0a graph=new Task0a(8);
```

```

graph.addEdge(0,1,1, false);
graph.addEdge(0,4,1, false);
graph.addEdge(1,2,1, false);
graph.addEdge(1,0,1, false);
graph.addEdge(2,3,1, false);
graph.addEdge(3,2,1, false);
graph.addEdge(3,4,1, false);
graph.addEdge(4,5,1, false);
graph.addEdge(5,6,1, false);
graph.addEdge(6,7,1, false);
graph.addEdge(7,6,1, false);
graph.addEdge(7,1,1, false);

graph.printWeight();
}

```

Adjacency List

```

public class EdgeNode{
    int toV;
    int weight;
    EdgeNode next;

    EdgeNode(int toV, int weight){
        this.toV=toV;
        this.weight=weight;
    }
}

```

```

public class Task0b {
    EdgeNode[] adjList;

    public Task0b(int n) {
        adjList = new EdgeNode[n];
    }
}

```



```

    }

    void addEdge(int from, int to, int weight, boolean undirected){
        EdgeNode n1= new EdgeNode(to, weight);
        n1.next= adjList[from];
        adjList[from]=n1;

        if(undirected==true){
            EdgeNode n2=new EdgeNode(from, weight);
            n2.next=adjList[to];
            adjList[to]= n2;
        }
    }

    void printList(){
        for(int i=0; i<adjList.length; i++){
            EdgeNode temp= adjList[i];
            while(temp!=null){
                System.out.print("(" + temp.toV + " , " + temp.weight+ "
)");

                temp=temp.next;
            }
            System.out.println();
        }
    }
}

```

```

public static void main(String[] args){
    Task0b graph0b=new Task0b(8);

    graph0b.addEdge(0,1,1, false);
    graph0b.addEdge(0,4,1, false);
    graph0b.addEdge(1,2,1, false);
    graph0b.addEdge(1,0,1, false);
    graph0b.addEdge(2,3,1, false);
    graph0b.addEdge(3,2,1, false);
    graph0b.addEdge(3,4,1, false);
}

```

```
graph0b.addEdge(4,5,1, false);  
graph0b.addEdge(5,6,1, false);  
graph0b.addEdge(6,7,1, false);  
graph0b.addEdge(7,6,1, false);  
graph0b.addEdge(7,1,1, false);  
  
graph0b.printList();  
}  
}
```

Lab Test

Q1. [10 marks] You are given a directed, weighted graph represented using an adjacency list. Your task is to find the **difference** between **the sum of the N largest edge weights** and **the sum of the N smallest edge weights** in the graph.

Write a function `difference_Sum(Edge[] adjList, int N)`, that takes a directed, weighted graph (in adjacency list representation) and an integer N (where $1 \leq N \leq$ total number of edges) as input parameters, and returns the required difference. **You can't use any data structures other than heap to solve this problem.**

Keep in mind, the adjacency list is given as an array of **Edge** objects and the **Edge** class has the following fields: **source (Type: String)**, **destination (Type: String)**, **weight (Type: Integer)**, **next (Type: Edge)**. Assume **MaxHeap** or **MinHeap** classes with the **insert(item)** and **extract()** (extract will return the extracted value) methods are already implemented for you.

Input	Output	Explanation
0 → (0, 1, 4) → (0, 2, 7) 1 → (1, 3, 3) → (1, 4, 6) 2 → (2, 3, 2) 3 → (3, 0, 1) → (3, 4, 5) 4 → () (no outgoing edges)	12	Edge Weights: {4, 7, 3, 6, 2, 1, 5} For N=3: • 3 largest weights = {7, 6, 5} → sum = 18 • 3 smallest weights = {1, 2, 3} → sum = 6 • Difference = 18 – 6 = 12

//Uses concepts from Graph + Heap

```
int difference_Sum(Edge[] adjList, int N) {
```

```
    MaxHeap maxH = new MaxHeap(N);
```

```
    MinHeap minH = new MinHeap(N);
```

```
    for (int i = 0; i < adjList.length; i++) {
```

```
        Edge curr = adjList[i];
```

```
        while (curr != null) {
```

```
            maxH.insert(curr.weight);
```

```
            minH.insert(curr.weight);
```

```
            curr = curr.next;
```

```
        }
```

```
    }
```

```
    int sumLargest = 0;
```

```
    int sumSmallest = 0;
```

```
    for (int i = 0; i < N; i++) {
```

```
        sumLargest += maxH.extract();
```

```
        sumSmallest += minH.extract();
```

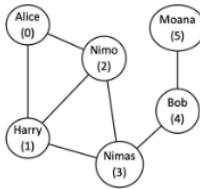
```
    }
```

```
    return sumLargest - sumSmallest;  
}
```

Rfts Sheet

You are given an **Undirected Unweighted Graph** representing a Facebook friendship network, where each vertex is a person and each edge is a friendship. Two people are said to have a mutual friend if there exists a third person who is directly connected to both of them. For example, Harry and Bob have one mutual friend, Nimas.

Your task is to write a function that finds the pair of distinct people that has the maximum number of mutual friends. If multiple pairs tie for the maximum, print any one of them. If no valid pair exists with at least one mutual friend, print the message, **"There are no mutual friends between any vertices."**



NOTE: The graph contains no self-loops or parallel edges. Vertex labels are mapped by a mapper array (Type: String).

The function/method `maxMutual()` takes a graph `g` represented in a 2D Matrix, and the mapper as a 1D Array of Strings in the parameter.

Sample Input	Sample Output
<pre> mapper= ["Alice","Harry","Nimo","Nimas","Bob","Moana"] g = 0 1 1 0 0 0 1 0 1 1 0 0 1 1 0 1 0 0 0 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 </pre>	Alice and Nimas have 2 mutual friend(s). Explanation: Alice and Nimas have 2 mutual friends, Harry and Nimo, which is the maximum in this graph.

Practise Problem

Function/Method Skeletons:

Python Notation

```
def maxMutual(g, mapper):
    # Write your code here
```

Java Notation

```
static void maxMutual(int [][] g, String[] mapper) {
    // Write your code here
}
```

14

```

public static void maxMutual(int[][] g, String[] mapper) {
    int n = g.length;
    int max = 0;
    int friend1 = -1, friend2 = -1;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {

            int mutualCount = 0;

            for (int k = 0; k < n; k++) {
                if (k != i && k != j && g[i][k] == 1 && g[j][k] == 1) {
                    mutualCount++;
                }
            }

            if (mutualCount > max) {
                max = mutualCount;
                friend1 = i;
                friend2 = j;
            }
        }
    }

    if (max == 0) {

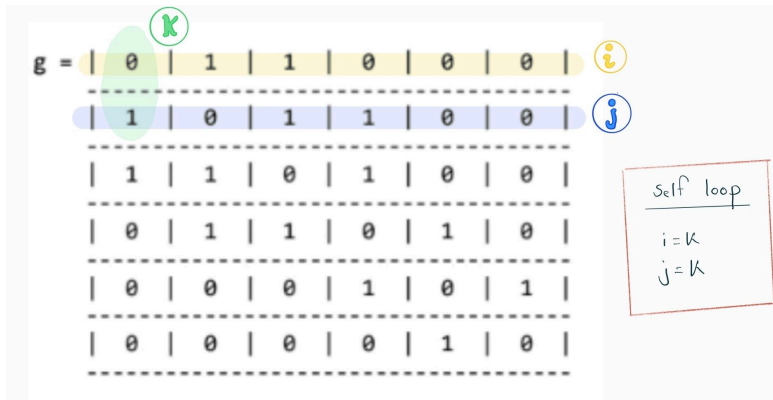
```

```

        System.out.println("There are no mutual friends between any vertices.");
    } else {
        System.out.println(
            mapper[friend1] + " and " + mapper[friend2] + " have " + max + " mutual friend(s)."
        );
    }
}

```

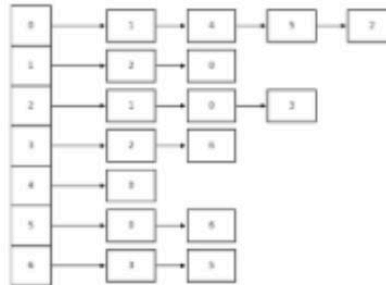
//More explanation



Write a function **areMutualFriends** that takes a graph represented as an adjacency list and determines whether two given persons **person1** and **person2** are mutual friends. Two persons are mutual friends if:

1. They **share at least one common friend**.
2. They are **not directly friends with each other**.

The function should return **True** (or **true**) if they are mutual friends, otherwise **False** (or **false**). Assume you are given a graph represented as an Adjacency List:



Sample Input	Sample Output
1. (person1 = 1, person2 = 4) 2. (person1 = 1, person2 = 6)	<pre> print(areMutualFriends(adjList, 1, 4)) # True print(areMutualFriends(adjList, 1, 6)) # False System.out.println(areMutualFriends(adjList, 1, 4)); // True System.out.println(areMutualFriends(adjList, 1, 6)); // False </pre> Explanation: In the first case (1, 4), Person 1 has neighbors {0, 2} and Person 4 has neighbors {0}. They share a common neighbor (0) but are not directly connected, so the result is True. In the second case (1, 6), Person 1 has neighbors {0, 2} and Person 6 has neighbors {3, 5}. They have no common neighbor, so the result is False.

```
boolean areMutualFriends(edgeNode[] adjList, int person1, int person2) {
```

```
    // Step 1: Check if they are directly friends
```

```
    edgeNode cur = adjList[person1];
```

```
    while (cur != null) {
```

```
        if (cur.toV == person2) {
```

```
            return false;           // directly friends
```

```
        }
```

```

    cur = cur.next;
}

// Step 2: Check for common friend
edgeNode p1 = adjList[person1];
while (p1 != null) {

    edgeNode p2 = adjList[person2];
    while (p2 != null) {

        if (p1.toV == p2.toV) {
            return true; // common friend found
        }

        p2 = p2.next;
    }

    p1 = p1.next;
}

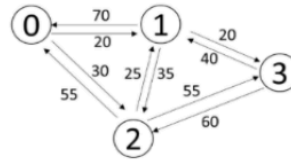
// Step 3: No common friend
return false;
}

```

Minimum incoming

The National Transport Authority monitors traffic volumes between cities every day. They are using a network modeled as a directed, weighted graph, where:

- Vertices represent cities
- Directed edges represent one-way roads
- Edge weights represent the average number of vehicles per day traveling from the source city to the destination city



The authority now wants to identify the city that has the least total incoming traffic, which refers to the vertex with the minimum sum of incoming edge weights. For the given example in the figure, the output should be city 1.

Now write a method/function that takes a directed, weighted graph (adjacency list) as an input parameter and returns the city with the minimum total incoming traffic volume. You can assume that the Edge class has source (Type: String), destination (Type: String), weight (Type: Integer), and a next (Type: Edge) variable. Note that the adjacency list represents the out-degree of the vertices.

Python Notation	Java Notation
<pre>def find_minimum(graph): #write your code here</pre>	<pre>static String find_minimum(Edge [] graph){ //write your code here }</pre>

Sample Input				Output	Explanation
0	1	2	3	1	The sum of the incoming edge weights for vertex 1 is: $20+25+40 = 85$ which is the minimum among all. Hence, 1 is returned.
<0,1,20>	<1,0,70>	<2,0,55>	<3,1,40>		
<0,2,30>	<1,2,35>	<2,1,25>	<3,2,60>		
<1,3,20>	<2,3,55>				

```

static String find_minimum(Edge[] graph) {

    int n = graph.length;

    // Step 1: incoming traffic array
    int[] incoming = new int[n];

    // Step 2: traverse all outgoing edges
    for (int i = 0; i < n; i++) {
        Edge curr = graph[i];

        while (curr != null) {
            int dest = Integer.parseInt(curr.destination);
            incoming[dest] += curr.weight;
            curr = curr.next;
        }
    }

    // Step 3: find vertex with minimum incoming sum
    int minIndex = 0;
    int minValue = incoming[0];

    for (int i = 1; i < n; i++) {
        if (incoming[i] < minValue) {
            minValue = incoming[i];
            minIndex = i;
        }
    }

    return String.valueOf(minIndex);
}
  
```

```

    }
}

return String.valueOf(minIndex);
}

```

A network graph is **complete** if every participant interacts with every other participant at least once.

Write a function `is_complete` to determine if a given network graph, represented as an unweighted adjacency list, is complete or not. If complete the function will return `True`, else `False`. **The `Edge` class has `source` (Type: `String`), `destination` (Type: `String`), and a `next` (Type: `Edge`) variable. You can use extra arrays if you want but cannot modify the given adjacency list.**

Python Notation	Java Notation
<pre>def is_complete(network): #To do</pre>	<pre>static boolean is_complete(Edge[] network){ //To do }</pre>

Input	Output	Explanation
<pre>network = [0: (0, 1) → (0, 2) → (0, 3), 1: (1, 0) → (1, 2) → (1, 3), 2: (2, 0) → (2, 1) → (2, 3), 3: (3, 0) → (3, 1) → (3, 2)]]</pre>	True	There are 4 participants: 0, 1, 2, and 3. Every participant has met every other participant at least once: - Participant 0 met with participants 1, 2, and 3. - Participant 1 met with participants 0, 2, and 3. - Participant 2 met with participants 0, 1, and 3. - Participant 3 met with participants 0, 1, and 2. Since every pair of participants has met at least once, the network is complete.
<pre>network = [0: (0, 1) → (0, 2) → (0, 3), 1: (1, 0) → (1, 2) → (1, 0), 2: (2, 0) → (2, 1) → (2, 3), 3: (3, 1) → (3, 2) → (3, 2)]]</pre>	False	There are 4 participants: 0, 1, 2, and 3. - Participant 1 has not met with participant 3. - Participant 3 has not met with participant 0. Since there are missing interactions, the network is incomplete.

```

static boolean is_complete(Edge[] network) {

    int n = network.length;

    for (int i = 0; i < n; i++) {
        boolean[] met = new boolean[n];
        met[i] = true;           // self
    }
}

```

```
Edge curr = network[i];
while (curr != null) {
    int v = Integer.parseInt(curr.destination);
    met[v] = true;
    curr = curr.next;
}

// check if i met everyone
for (int j = 0; j < n; j++) {
    if (met[j]==false) return false;
}
}

return true;
}
```